

Real-Time, Events & Reliability

The end of Part V (real-time & event-driven) and the start of Part VI (reliability).

31 Real-Time Communication — polling, SSE, and WebSockets

32 Event-Driven Architecture — reacting to things that happened

33 Event Sourcing & CQRS — events as truth, separate read/write models

34 Reliability, Availability & Redundancy — nines, SLAs, no single point of failure

35 Rate Limiting & Throttling — protecting a system from overload

Chapters 31–33 finish how systems communicate; 34–35 begin how they survive failure and overload.

CHAPTER 31

Real-Time Communication

Part V · Communication & Architecture — pushing data to clients as it happens.

LEARNING OBJECTIVES

- Explain why plain HTTP request-response can't push data to clients.
- Compare short polling, long polling, SSE, and WebSockets.
- Choose the right technique for a use case.
- Explain why real-time connections are stateful and how to scale them.

REAL-WORLD ANALOGY — ASKING FOR MAIL VS BEING CALLED

You can walk to the front desk every few minutes asking “any mail?” (polling), or the desk can phone you the moment something arrives (push). Better still, you can keep a phone line open — a news ticker one way (SSE) or a full two-way call (WebSocket). Real-time communication is about getting the server to reach the client, not just the other way around.

31.1 The problem

HTTP is **client-initiated** request-response (Chapter 4): the server can only reply when asked. But chat, notifications, live scores, and collaborative editing need the **server to push** data to the client the instant it changes. Several techniques bridge this gap.



31.2 The four options

Technique	How it works	Best for
Short polling	Client re-asks every few seconds.	Simple, infrequent updates; wasteful and laggy.
Long polling	Server holds the request open until data (or timeout), then client re-asks.	Near-real-time fallback where WebSockets aren't available.
SSE	One long-lived HTTP connection; server streams events one way .	Live feeds, notifications, dashboards (server→client only).

WebSocket

Persistent **full-duplex** connection upgraded from HTTP.

Chat, games, collaborative editing (interactive two-way).

31.3 Scaling real-time connections

KEY INSIGHT — PERSISTENT CONNECTIONS ARE STATEFUL

Unlike stateless HTTP requests (Chapter 6), a WebSocket/SSE connection **stays open** and lives on one specific server, consuming memory per connection. That reintroduces state: you need connection management, careful routing, and — crucially — a **pub/sub backplane** (e.g. Redis, Chapters 8/28) so that a message produced on one server can reach clients connected to *other* servers. Millions of concurrent connections require infrastructure built for exactly this.

31.4 Choosing

Use **short polling** for simple, infrequent updates; **long polling** as a fallback; **SSE** for one-way server streams; and **WebSockets** when you truly need interactive, bidirectional real-time.

Common mistakes

WATCH OUT

- Short-polling aggressively and drowning the server in empty requests.
- Using WebSockets where one-way SSE (or even polling) would suffice.
- Forgetting real-time connections are stateful — no backplane to broadcast across servers.
- Ignoring reconnection, heartbeats, and connection limits.

Interview questions

1. Why can't plain HTTP push data to a client?
2. Compare short polling, long polling, SSE, and WebSockets.
3. When would you pick SSE over WebSockets?
4. Why are real-time connections hard to scale, and how do you broadcast across servers?

Hands-on exercises

1. Pick a technique for: stock ticker, two-player game, deploy-status page, group chat.
2. Estimate the wasted requests of 1s short polling for an idle user over an hour.
3. Explain how a Redis pub/sub backplane lets two WebSocket servers share a broadcast.
4. Describe how a client should reconnect after a dropped SSE stream.

Mini project

BUILD A LIVE FEED THREE WAYS

Implement a “live counter” that updates in the browser using (1) short polling, (2) SSE, and (3) WebSockets. Compare request counts, latency, and code complexity. Then run two server instances and add a Redis pub/sub backplane so an update on one reaches clients on the other.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — REAL-TIME FEATURES

Decide the transport for each real-time feature: notifications and live feed updates (SSE), direct messaging and typing indicators (WebSockets). Plan the **backplane** so a message reaches a recipient regardless of which server holds their connection, and set per-server connection limits from your Chapter 5 numbers.

Chapter summary

- Plain HTTP is client-initiated; real-time needs server **push**.
- **Short/long polling** approximate it over request-response; **SSE** streams one way; **WebSockets** are full-duplex.
- Persistent connections are **stateful** and per-server — scaling needs a **pub/sub backplane** and connection management.
- Match the technique to the interaction: polling → SSE → WebSockets as needs grow.

CHAPTER 32

Event-Driven Architecture

Part V · Communication & Architecture — reacting to things that happened.

LEARNING OBJECTIVES

- Explain event-driven architecture and how it differs from request-driven.
- Distinguish events from commands.
- Compare choreography and orchestration.
- Know the benefits and the challenges (consistency, tracing).

REAL-WORLD ANALOGY — A NEWSROOM

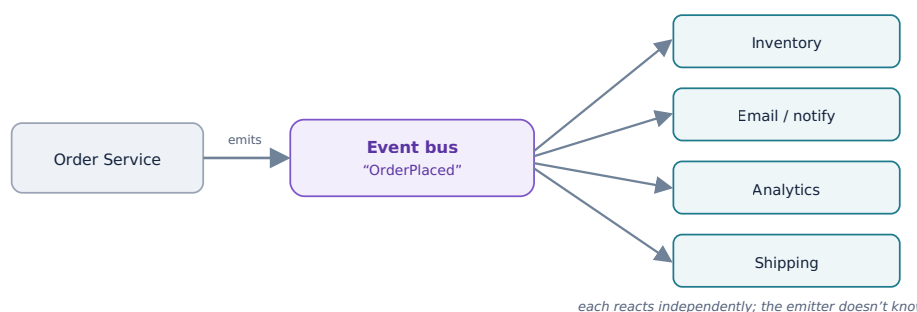
Reporters publish stories (events); anyone interested subscribes and reacts — the sports desk, the archive, the website — without the reporter phoning each of them. New consumers can start following the feed without the reporter ever knowing. Event-driven systems work the same way: services announce facts, and others react.

32.1 The problem

In request-driven systems, a service directly calls the others it needs, so it must know about them — tight coupling (Chapters 29–30). **Event-driven architecture** inverts this: a service emits an **event** (“something happened”) and others react, with the emitter unaware of who listens. Built on messaging (Chapter 28), it decouples producers from consumers.

32.2 Events vs commands

A **command** says “do X” — directed at a specific handler, expecting an action. An **event** says “X happened” — a past-tense fact, broadcast to whoever cares. Event-driven systems are built on events, not commands.



32.3 Benefits

Loose coupling (the emitter doesn't know its consumers), **extensibility** (add a new reacting service without touching the emitter), **async and scalable** (Chapter 28), and **resilient** (events are buffered if a consumer is down).

32.4 Choreography vs orchestration

Choreography	Orchestration
Services react to each other's events with no central controller. Decentralized and flexible — but the overall flow is implicit and harder to see or debug.	A central orchestrator directs the workflow, telling each service what to do. Clear control and visibility — but the orchestrator is a coupling point and can grow complex.

32.5 Challenges

THE COST OF DECOUPLING

Event-driven systems are **eventually consistent** (Chapter 21), and a business flow is now spread across many independent reactions — so **tracing and debugging** require good observability (Chapter 50). You must also handle event **schema evolution** and, from Chapter 28, duplicate delivery and ordering. Power comes with operational complexity.

Common mistakes

WATCH OUT

- Emitting commands disguised as events (coupling the emitter to a specific handler).
- All-choreography flows that no one can trace end to end.
- Ignoring eventual consistency and assuming reactions are instantaneous.
- No plan for event versioning or duplicate handling.

Interview questions

1. What is event-driven architecture, and how does it differ from request-driven?
2. Contrast an event with a command.
3. Compare choreography and orchestration.
4. What are the main challenges of event-driven systems?

Hands-on exercises

1. List four services that might react to an `OrderPlaced` event.
2. Rewrite a direct “order service calls email service” call as an event.
3. Decide choreography or orchestration for a multi-step checkout, and justify.
4. Describe how you'd trace a single order across all its reactions.

Mini project

EMIT AND REACT

Build a tiny event-driven flow: an order service publishes `OrderPlaced` to a bus; separate consumers for inventory, email, and analytics each react independently. Add a fourth

consumer later *without changing the emitter*. Then add a correlation ID to trace one order through all reactions.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE EVENT MODEL

Define your core domain events (`PostCreated` , `UserFollowed` , `PostLiked`) and the services that react to each (feed fan-out, notifications, analytics, search index). Choose choreography vs orchestration for the post-creation flow, and plan correlation IDs so you can trace an event's ripple across services.

Chapter summary

- **Event-driven architecture** has services emit **events** (facts) that others react to, decoupling producers from consumers.
- **Events** ("X happened") differ from **commands** ("do X").
- **Choreography** (no central control) vs **orchestration** (a central coordinator) — a visibility/coupling trade-off.
- Benefits: loose coupling, extensibility, resilience — at the cost of eventual consistency and harder tracing.

CHAPTER 33

Event Sourcing & CQRS

Part V · Communication & Architecture — events as the source of truth.

LEARNING OBJECTIVES

- Explain event sourcing and how state is derived from events.
- Explain CQRS and why read and write models are separated.
- Explain why the two patterns pair naturally.
- Decide when they're worth the complexity.

REAL-WORLD ANALOGY — A LEDGER, NOT JUST A BALANCE

A bank doesn't store only your current balance — it stores every deposit and withdrawal, and the balance is *derived* by adding them up. Keeping the full list of transactions means you can audit, replay, and reconstruct the balance at any past moment. **Event sourcing** stores the events; the current state is just their sum.

33.1 The problem

Traditional CRUD stores only **current state**, so history is lost, and one model must serve both writes (validation, consistency) and complex reads (fast, varied queries) — a constant tension. Two related patterns address this.

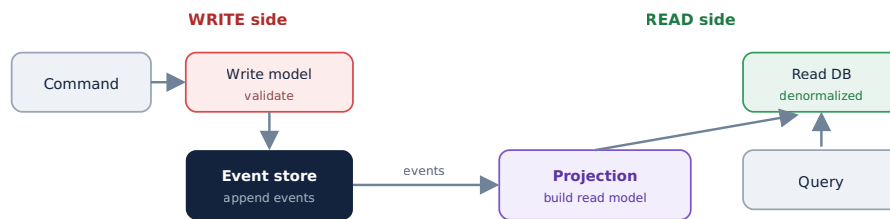
33.2 Event sourcing

Instead of storing current state, store the full, ordered log of **events** that changed it — the events are the **source of truth**. Current state is obtained by **replaying** them. Benefits: a complete **audit log** and history, the ability to reconstruct state at any point ("time travel"), and a natural fit with event-driven systems (Chapter 32). Costs: more complexity, event **schema evolution**, and replay cost (mitigated with periodic **snapshots**).

33.3 CQRS

Command Query Responsibility Segregation splits the **write model** (commands — optimized for validation and consistency) from the **read model** (queries — denormalized and optimized for fast reads, Chapter 13). The two can be separate data stores, kept in sync. Benefit: reads and writes scale and optimize independently. Cost: added complexity and **eventual consistency** between the write and read sides.

33.4 The two together



writes append events; read models (projections) are built by consuming those events

KEY INSIGHT — A NATURAL PAIRING

Event sourcing and CQRS fit together: the write side **appends events** to the event store, and one or more read-side **projections** consume those events to build fast, query-optimized read models. You get a full history *and* tailored read views — at the price of eventual consistency and real complexity.

33.5 When to use them

Reach for these in **audit-heavy** domains (finance, orders, anything needing a full history), genuinely **complex** domains, or when read and write needs diverge sharply. For simple CRUD they are **over-engineering** — don't adopt them by default.

Common mistakes

WATCH OUT

- Applying event sourcing/CQRS to a simple CRUD app that doesn't need them.
- Ignoring eventual consistency between the write and read sides.
- No snapshot strategy, so replaying millions of events is slow.
- No plan for evolving event schemas over time.

Interview questions

1. What is event sourcing, and how is current state derived?
2. What is CQRS, and why separate read and write models?
3. Why do the two patterns pair well?
4. What are the costs, and when are they justified?

Hands-on exercises

1. Model a bank account with event sourcing: list the events and how balance is derived.
2. Design separate write and read models for a “posts with like counts” feature.
3. Explain how a snapshot speeds up replay.
4. Decide whether a simple to-do app should use event sourcing, and justify.

Mini project

EVENT-SOURCE A SMALL AGGREGATE

Implement an account as an event-sourced aggregate: append `Deposited / Withdrew` events and derive the balance by replay. Add a CQRS read model — a projection that maintains a fast “current balance” table by consuming events. Add a snapshot and measure the replay speedup.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — SELECTIVE EVENT SOURCING

Decide where event sourcing earns its keep — likely a small, audit-worthy slice such as account/billing or moderation actions — and keep the rest as normal CRUD. If you adopt CQRS for the feed, define the write model (append events) and the read projection (a denormalized feed table), and note the eventual-consistency window.

Chapter summary

- **Event sourcing** stores the log of events as truth; current state is derived by **replay** (with snapshots to speed it up).
- **CQRS** separates the **write** model from denormalized **read** models for independent optimization.
- They **pair naturally**: writes append events, projections build read models — full history plus fast reads.
- Powerful for audit-heavy/complex domains; **over-engineering** for simple CRUD.

CHAPTER 34

Reliability, Availability & Redundancy

Part VI · Reliability & Resilience — designing to survive failure.

LEARNING OBJECTIVES

- Define reliability and availability, and read availability in “nines.”
- Explain SLI, SLO, and SLA and error budgets.
- Explain redundancy and eliminating single points of failure.
- Explain why availability is a chain, and how redundancy raises it.

REAL-WORLD ANALOGY — AN AIRCRAFT’S BACKUPS

An airplane has redundant engines, hydraulics, and computers precisely so that *no single failure* brings it down. It’s designed on the assumption that parts **will** fail. Reliable systems take the same stance: failures are inevitable (Chapter 19), so build in backups and remove any part whose loss stops everything.

34.1 Part VI begins

We’ve built and connected systems; now we make them **survive failure**. Part VI is reliability engineering: redundancy, timeouts and retries, circuit breakers, rate limiting, graceful degradation, and more. It starts with the vocabulary and the core technique: redundancy.

34.2 Reliability and availability

Reliability is whether the system works correctly and keeps working. **Availability** is the fraction of time it’s up, usually stated in **nines**:

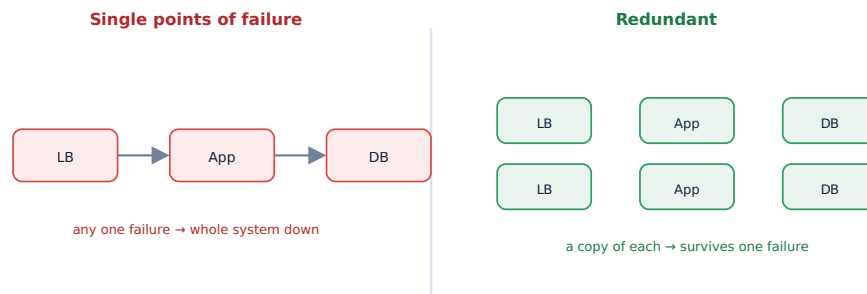
Availability	Downtime per year
99% (“two nines”)	~3.65 days
99.9% (“three nines”)	~8.8 hours
99.99% (“four nines”)	~52 minutes
99.999% (“five nines”)	~5 minutes

Availability $\approx$ MTBF / (MTBF + MTTR) — so cutting **recovery time** (MTTR) raises availability just as much as failing less often. Fast recovery is a design goal.

34.3 SLI, SLO, SLA

SLI (indicator) is a measured metric — error rate, p99 latency (Chapter 5). An **SLO** (objective) is your internal target for it (e.g. 99.9% success). An **SLA** (agreement) is a contractual promise to customers, with penalties. The gap between 100% and your SLO is the **error budget** — how much unreliability you can “spend” on releases and risk.

34.4 Redundancy and single points of failure



The core technique is **redundancy**: duplicate every critical component — servers, load balancers (Chapter 7), databases (Chapter 16), even regions — so no single failure takes the system down. Configurations include **active-active** (all serving) and **active-passive** (a standby takes over). A component whose loss stops everything is a **single point of failure (SPOF)** to be eliminated.

34.5 Availability is a chain

KEY INSIGHT — DEPENDENCIES MULTIPLY DOWN; REDUNDANCY MULTIPLIES UP

When components are in **series** (each needed for a request), overall availability is the **product** of theirs — three 99.9% dependencies give ~99.7%, worse than any one. So every dependency on the critical path *lowers* availability. **Redundancy** puts components in **parallel**: two 99% replicas fail together only 0.01% of the time → 99.99%. Reduce critical-path dependencies, and add redundancy where you can't.

Common mistakes

WATCH OUT

- Leaving a single load balancer, database primary, or region as a SPOF.
- Chasing “more nines” while ignoring MTTR (slow recovery).
- Adding many serial dependencies and eroding availability without noticing.
- Promising an SLA stricter than the architecture can deliver.

Interview questions

1. Define reliability and availability. What does 99.99% mean in downtime?
2. Explain SLI, SLO, SLA, and error budgets.
3. How does redundancy remove single points of failure? Active-active vs active-passive?
4. Why is availability a product of dependencies, and how does redundancy help?
5. Why does reducing MTTR improve availability?

Hands-on exercises

1. Convert 99.9% and 99.99% to downtime per month.
2. Find the SPOFs in a single-LB, single-DB architecture and fix each.

3. Compute the availability of three 99.9% dependencies in series.
4. Set an SLI, SLO, and SLA for a checkout API.

Mini project

FIND AND REMOVE A SPOF

Take a simple deployment (one LB, two app servers, one DB). Identify every single point of failure. Add a redundant load balancer and a database replica with failover. Then simulate killing each component and confirm the system stays up — measuring the recovery time (MTTR) for each failure.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE AVAILABILITY TARGET

Set an availability **SLO** (e.g. 99.9%) and derive the redundancy it demands: redundant LBs (Chapter 7), DB replicas with failover (Chapter 16), multi-AZ (and later multi-region). List every current SPOF and its fix, define your key SLIs, and note where reducing MTTR (faster failover) is cheaper than adding another nine.

Chapter summary

- **Reliability** = works correctly; **availability** = fraction of uptime, measured in **nines**.
- $\text{Availability} \approx \text{MTBF}/(\text{MTBF} + \text{MTTR})$ — **faster recovery** raises it, not just fewer failures.
- **SLI** (metric), **SLO** (target), **SLA** (contract); the slack is your **error budget**.
- **Redundancy** removes **single points of failure**; availability is a **product** of serial dependencies and improved by **parallel** redundancy.

CHAPTER 35

Rate Limiting & Throttling

Part VI · Reliability & Resilience — protecting a system from overload.

LEARNING OBJECTIVES

- Explain why rate limiting protects a system and its users.
- Compare fixed window, sliding window, token bucket, and leaky bucket.
- Explain where to apply limits and how to do it in a distributed system.
- Respond to limited clients correctly.

REAL-WORLD ANALOGY — A TURNSTILE

A venue puts a turnstile at the entrance that admits only so many people per minute. It doesn't matter how big the crowd outside is — the flow inside stays safe and orderly. **Rate limiting** is that turnstile for requests: it caps how fast any client can enter so the system behind stays healthy.

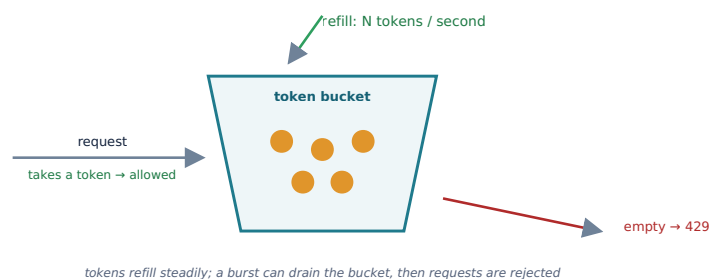
35.1 The problem

Unbounded requests can overwhelm a system (recall the utilization curve, Chapter 5), enable abuse and denial-of-service, trigger cascading failures, and run up costs. **Rate limiting** caps how many requests a client may make in a time window — a first-class protection, usually enforced at the edge (the API gateway, Chapter 9).

35.2 Why limit

To **protect resources** from overload, ensure **fair usage** among clients, prevent **abuse and DoS**, **control cost**, and enforce **quotas/tiers** (free vs paid).

35.3 The algorithms



Algorithm	How it works
Fixed window	Count requests per fixed interval (100/min). Simple, but allows bursts at window edges.
Sliding window	Smooths the edge problem with a rolling window/log.

Token bucket	Tokens refill at a steady rate; each request spends one; allows bursts up to bucket size. The popular, flexible default.
Leaky bucket	Requests queue and drain at a fixed rate — smooths the <i>output</i> flow.

35.4 Where and how to apply

Apply limits at the **edge** (API gateway), keyed per **user / IP / API key**, and often per endpoint. In a distributed system with many gateway instances, the counter must be **shared** — typically an atomic counter in **Redis** (Chapter 8) — so limits are enforced globally, not per instance.

35.5 Responding to limited clients

Return **HTTP 429 Too Many Requests** (Chapter 4) with a **Retry-After** header so clients know when to try again, and document limits clearly. **Throttling** is a gentler cousin: instead of rejecting, slow the client down — useful when you'd rather degrade than deny.

Common mistakes

WATCH OUT

- Per-instance counters that let clients exceed the global limit by hitting different servers.
- Fixed windows that allow a double burst across the window boundary.
- Rejecting without **429 / Retry-After**, leaving clients to hammer blindly.
- One global limit instead of per-user/per-key fairness.
- Limits so tight they block legitimate traffic (or so loose they don't protect).

Interview questions

1. Why rate-limit, and what does it protect against?
2. Compare fixed window, sliding window, token bucket, and leaky bucket.
3. How do you enforce a limit across many gateway instances?
4. How should a server respond to a rate-limited client?
5. What's the difference between throttling and rate limiting?

Hands-on exercises

1. Show how a fixed window allows a burst of 2× the limit at the boundary.
2. Trace a token bucket (size 5, refill 1/s) through a burst of 8 requests.
3. Design a distributed limiter using a Redis atomic counter.
4. Choose limits and keys for a public API with free and paid tiers.

Mini project

BUILD A TOKEN-BUCKET LIMITER

Implement a token-bucket rate limiter (configurable size and refill rate) as middleware. Return **429** with **Retry-After** when the bucket is empty. Then make it distributed with a Redis atomic counter so two server instances share one limit, and load-test to confirm the global cap holds.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE LIMITS

Define rate limits at your gateway: per-user posting and following limits (anti-spam), per-IP limits on auth endpoints (anti-abuse), and tiered limits for any public API. Choose the algorithm (token bucket), the key (user/IP/API key), and a shared Redis counter for distributed enforcement. Specify the 429 + Retry-After response.

Chapter summary

- **Rate limiting** caps request rate to protect resources, ensure fairness, prevent abuse, and control cost.
- Algorithms: **fixed/sliding window**, **token bucket** (flexible, allows bursts), **leaky bucket** (smooths output).
- Apply at the **edge**, keyed per user/IP/key; use a **shared (Redis) counter** for distributed limits.
- Respond with **429 + Retry-After**; **throttling** slows rather than rejects.

COMING NEXT

Chapters 36-40 continue Part VI: timeouts, retries & backoff; circuit breakers & bulkheads; Sagas & distributed consistency; Bloom filters & probabilistic structures; and graceful degradation & chaos engineering.